

# Report

## 1. System's Perspective

### 1.1

### 1.2 Dependencies of MiniTwit

| Technology                          | Stage                            | Role                                 |
|-------------------------------------|----------------------------------|--------------------------------------|
| Git / GitHub                        | Development, CI/CD               | Source control, reviews, and work tr |
| Trello                              | Development, operations          | Backlog management and work tr       |
| Discord                             | Development, operations          | Team communication and receivin      |
| C# / .NET                           | Development, production          | Application language and runtime     |
| NuGet                               | Development, CI/CD               | Package restore and feeds for .NE    |
| ASP.NET Core (Razor Pages)          | Development, production          | Web UI and HTTP API                  |
| Entity Framework Core               | Development, production          | Database access and migrations       |
| PostgreSQL                          | Development, testing, production | Primary data store (managed in I     |
| Docker                              | Development, CI/CD, production   | Container images and runtime iso     |
| Docker Hub                          | CI/CD, production                | Registry for built application ima   |
| Docker Compose                      | Development, testing             | Local and test multi-container set   |
| Docker Swarm                        | Production                       | Orchestration and rolling updates    |
| DigitalOcean                        | Infrastructure, production       | Cloud VMs, managed database, a       |
| DigitalOcean Spaces (S3-compatible) | Infrastructure, CI/CD            | Object storage with an S3-compa      |
| Terraform                           | Infrastructure, CI/CD            | Infrastructure as code for managi    |
| GitHub Actions                      | CI/CD                            | Continuous integration and deplo     |
| Third-party GitHub Actions          | CI/CD                            | Marketplace and vendor-maintain      |
| GitHub CLI                          | CI/CD                            | Command-line GitHub integratio       |
| Ubuntu                              | CI/CD, production                | Base operating system on GitHub      |
| SSH (OpenSSH)                       | CI/CD, production                | Remote deploy, file copy, and serv   |
| Prometheus                          | Monitoring                       | Metrics collection                   |
| Grafana                             | Monitoring                       | Dashboards and alerting              |
| Loki                                | Monitoring                       | Centralized log storage              |
| Promtail                            | Monitoring                       | Shipping container logs to Loki      |
| Python                              | Testing (local and CI/CD)        | API simulator test driver and tes    |
| Selenium                            | Testing (local and CI/CD)        | Browser-based UI tests (with Chr     |
| dotnet format                       | Development, CI/CD               | C# formatting and verifying the      |
| Roslynator                          | Development, CI/CD               | C# static analysis (Roslyn-based     |
| Codespell                           | Development, CI/CD               | Spell checking across the reposito   |
| Hadolint                            | Development, CI/CD               | Linting Dockerfiles                  |
| Codacy                              | Development, CI/CD               | Hosted static analysis and pull-re   |
| SonarCloud                          | Development, CI/CD               | SonarQube-family analysis and qu     |
| CodeQL                              | Development, CI/CD               | Semantic security and quality sca    |
| Docker Scout                        | CI/CD                            | Container image vulnerability sca    |
| OpenAPI Generator                   | Development                      | Generating the API simulator stu     |
| Pandoc / LaTeX                      | CI/CD                            | Building the report PDF in autor     |
| GNU Make                            | Development, CI/CD               | Task automation (local and in wo     |

**Libraries** NuGet references come from the Razor Pages solution (`razor-pages/Web`, `razor-pages/Infrastructure`). Direct Python libraries for automated tests are listed below; the UI test lockfile also pins transitive versions (see `tests/selenium/requirements.txt`).

| Library                                           | Context             | Usage                                        |
|---------------------------------------------------|---------------------|----------------------------------------------|
| Microsoft.AspNetCore.Identity.EntityFrameworkCore | Web, Infrastructure | ASP.NET Core Identity integration            |
| Microsoft.EntityFrameworkCore.Design              | Web, Infrastructure | EF Core design-time support and scaffolding  |
| Npgsql.EntityFrameworkCore.PostgreSQL             | Web, Infrastructure | EF Core provider for PostgreSQL              |
| Newtonsoft.Json                                   | Web                 | JSON serialization and deserialization       |
| Swashbuckle.AspNetCore.Annotations                | Web                 | OpenAPI metadata and attributes              |
| Swashbuckle.AspNetCore.Newtonsoft                 | Web                 | OpenAPI generation using Newtonsoft.Json     |
| DotNetEnv                                         | Web                 | Loading environment variables                |
| prometheus-net.AspNetCore                         | Web                 | Prometheus metrics for ASP.NET               |
| Microsoft.CodeAnalysis.Analyzers                  | Infrastructure      | Build-time Roslyn analyzers                  |
| Microsoft.Extensions.Hosting                      | Infrastructure      | Hosting abstractions for background services |
| prometheus-net                                    | Infrastructure      | Prometheus metric registration               |
| Npgsql                                            | Infrastructure      | PostgreSQL data provider (ADO.NET)           |
| TimeZoneConverter                                 | Infrastructure      | Resolving time zones in infrastructure       |
| requests                                          | tests/API_Spec      | HTTP calls from the API simulation           |
| pytest                                            | tests/selenium      | Test runner for the Selenium UI tests        |
| selenium                                          | tests/selenium      | WebDriver client driving the browser         |

### 1.3

## 2. Process' perspective

### 2.1 CI/CD pipelines, deployment, and release

All development work is done on branches and requires a pull request to be merged into the main branch. Pull Requests are automatically checked with code scanning tools and also triggers a QA build which runs a full build, test and deployment to a QA droplet and database. Note, that due to limitations on number of allowed droplets in our Digital Ocean account level, this QA droplet was later included in the production swarm as well.

After merging a pull request into main, the report pdf is built iff changes have been made in the relevant files. Nothing is immediately pushed to production as we deemed that we wanted our releases to contain more than a single small change and have more control of when releases to production were made. The control of timing is important to ensure stability of the application and timely action given a failure/bug.

We used an automated deployment pipeline to deploy our production services that automatically triggers when a tag is pushed to the repository. We attempted to follow a form of semantic versioning for tag names, to have a consistent format

and a notion of how big each release was. The automated deployment builds a docker image and deploys the stack on the swarm leader node.

Monitoring is deployed manually in a separate workflow. The monitoring droplet was initially a stand-alone droplet, but given the Digital Ocean limitation on droplets, this droplet was also later included in the swarm. The monitoring deployment could have been automatically deployed if changes appeared in the relevant root folder, yet changes to the configurations were rather rare and we therefore did not find it necessary.

Below is an overview of the different stages of development towards operationalization. In the following sections we will deep dive into the QA deployment workflow, continuous deployment release workflow and the monitoring deployment workflow.

```
flowchart LR
    subgraph dev [Development Branch]
        A[Branch Work] --> B[Make a PR]
        B --> C[QA Deployment workflow]
        B --> K[Automated Checks]
        C --> L[Peer Review]
        K --> L[Peer Review]
    end
    end
    subgraph merge [Main Branch]
        L --> D[Merge to main]
        D --> E[Report build on report changes]
    end
    end
    subgraph release [Release Tagging]
        D --> F[Semantic version tag]
        F --> G[Continuous Deployment]
        G --> H[Docker Swarm production]
    end
    end
    subgraph ops [Operationalization]
        D --> I[Deploy Monitoring manual]
        I --> J[Prometheus / Grafana / Loki]
        H --> J
    end
    end
```

**Pull-request pipeline (QA Deployment)** The QA deployment is defined in `.github/workflows/continous-QA-deployment.yaml` and is automatically run on pull requests towards the main branch.

```
sequenceDiagram
    participant GH as GitHub Actions
    participant Hub as Docker Hub
    participant QA as QA droplet
    participant DB as PostgreSQL QA DB
```

```

GH->>Hub: build and push testminitwit
GH->>GH: terraform plan + PR comment
GH->>QA: scp env + compose-test.yaml
GH->>GH: Docker Scout CVE scan
GH->>QA: Deploy Docker Compose
Hub->>QA: Pull Latest Image
QA->>QA: Docker Compose Up
GH->>DB: reset schema / simulator state
GH->>GH: run-all-validations
GH->>QA: API simulator + Selenium against :8081
QA->>DB: Query
DB->>QA: Data
QA->>GH: API / UI Results

```

Above flow chart shows the various steps and interactions between systems happening during the QA Deployment and test workflow. The workflow runs at the same time as the static code analysis tools CodeQL, SonarCube and Codacy.

**Production release (Continuous Deployment)** The Continuous deployment to production is defined in `.github/workflows/continous-deployment.yaml` and is automatically run on tags pushed to the main branch.

```

sequenceDiagram
    participant GH as GitHub Actions
    participant Hub as Docker Hub
    participant DO as DigitalOcean
    participant Mgr as Swarm manager
    participant Nodes as Swarm cluster
    participant DB as Managed PostgreSQL

    GH->>Hub: build and push minitwitimage:SHA
    GH->>DO: terraform apply prod
    Note over DO,DB: Applies IaC for VMs network and managed Postgres etc.
    GH->>Mgr: scp compose.yaml
    GH->>Mgr: docker stack deploy minitwit
    Hub->>Nodes: pull image (with-registry-auth)
    Mgr->>Nodes: rolling update (3 replicas)

```

**Monitoring deployment** The monitoring stack deployment is defined in `.github/workflows/monitor-deployment.yaml` and runs only when someone manually triggers it.

```

sequenceDiagram
    participant GH as GitHub Actions
    participant Hub as Docker Hub
    participant Mon as Monitoring server

```

participant Mgr as Swarm manager

```
GH->>Mon: scp repo monitoring/* to /vagrant/monitoring
GH->>Mgr: scp repo monitoring/* to /vagrant/monitoring
GH->>Mgr: docker stack deploy monitoring
Mgr->>Mon: deploy constrained to server
Hub->>Mon: Pull Prometheus, Grafana and Loki Images
Mon->>Mon: Run Prometheus, Grafana and Loki Containers
```

## Deployment & Release Summary

| Environment    | How it is updated                                                         | Image / orchestration                                 |
|----------------|---------------------------------------------------------------------------|-------------------------------------------------------|
| Local          | <code>make app-build</code> ( <code>compose-test.yaml</code> , port 8081) | Local Docker Compose                                  |
| QA (pre-merge) | QA Deployment workflow on PR                                              | <code>testminitwit:latest</code> , Compose on t       |
| Production     | Tag → Continuous Deployment                                               | <code>minitwitimage:&lt;sha&gt;</code> , Docker Swarm |
| Monitoring     | Manual Deploy Monitoring workflow                                         | Swarm stack monitoring                                |

## 2.2

### 2.3 Aggregated logs

All assignment completions for each week have been aggregated in View project log . It was standard practice for everyone to document which tasks they completed. A type of "Meta" log used is the README file it serves as how we ought to implement the assignments as well as principle on how work as a group Docker has a built-in log system for each droplet. This logging system was rarely used except for some debugging cases. all live logs are shipped to grafana alt text

## 2.4

### 2.5 Availability and Scaling

Availability and scaling in the MiniTwit application are managed by Docker Swarm. A Swarm cluster of the DigitalOcean Droplets is joined into a single Swarm cluster, which continuously monitors and enforces the declared desired state.

**High availability** is handled by having manager redundancy, three production container replicas, and automatic self-healing. All three nodes in the cluster are given the `manager` role to prevent a single point of failure if one of the manager nodes crashes. When a node in the cluster crashes, the Swarm detects a difference between the actual state and the declared desired state, such that the number of actual running replicas is lower than three, which triggers *self-healing* to restore the third replica.

**Scaling** is handled by Docker Swarm's built-in Ingress Routing Mesh which functions as a load balancer. Swarm evenly distributes incoming user requests across all three healthy replicas of the production container to handle high amounts of concurrent requests. This parallelizes the workload across the nodes, such that a container does not consume all the resources of a single node.

To ensure low downtime during the transition from the standalone containers to a Docker Swarm cluster, the stack was deployed with a **blue-green service deployment** strategy from the start, such that the running containers were gradually replaced by the updated ones. This is achieved by configuring the deployment settings within the Docker Compose file to set the update order to **start-first** and a delay parameter that dictates how long Docker Swarm should wait after starting an updated container before terminating an old one, allowing the new container to initialize.

As a result, the services remain available during deployment. By default, Docker Swarm uses the rolling update strategy which terminates the old container before starting a new one. This is called **stop-first**. By terminating the containers first, the default strategy forces the application to experience downtime during the window between container termination and container initialization.

While our strategy should have ensured low downtime during the transition to Docker Swarm, the production application still experienced downtime due to overlooked human errors. These errors came as a result of debugging separate Docker Swarm migration issues. Specifically, the Swarm Ingress routing mesh overwrote the port of the development environment on one of our DigitalOcean Droplets, and the Loki logs failed to display on our Grafana dashboards. During the debugging process, accidental downtime of the application was introduced when pulling the wrong container image due to misconfigured environment secrets, or changing the application to use another port, which prevented the simulator from reaching it.

To avoid these issues in the future, a solution could be to replicate the Docker Swarm infrastructure within an isolated development environment, so any configuration changes during the transition does not affect live production.

### 3. Reflection Perspective

#### 3.1 Evolution and Refactoring

On first refactoring from Pyhton to RazorPages with C# we ran into unforeseen issues with the methods not working as intended. This slowed us down but once the bugs were solved we were able to make our release. We had no issues Refactoring to our Onion Structure. It was time-consuming but that half the group being familiar with the framework made the proces smooth.

### 3.2 Operation

### 3.3 Maintenance

## 4. Use of Generative AI

The generative AI tool Cursor was used to discuss issues and warnings throughout the project and provide guidance on issues that we had not encountered before. This was beneficial to the development process as it unblocked us in completing tasks. We used Cursor to improve the code in terms of maintainability by standardizing the format and structure along with industry standard formatting / linting tools (i.e. using a mix of CLI tools and GenAI). We also employed Cursor to summarize branch work in our log.md and other documentation, to ensure chore tasks were being done rather than neglected. These logs and documentation are used to remind us of the work and considerations throughout the different stages of the project which have been written / read for internal use.

ChatGPT was used for debugging GitHub Actions, Docker, and DevOps setup issues, it often hindered our work on the Deployment workflows but helped significantly on Command lines in the terminal.

The AI tool Github Copilot was used throughout the course for better code understanding, serving as interactive documentation. It was also used for very specific code fixes, usually hinted by SonarQube, such as `b6f71cf3242a25a0c03cd0c0763040417532838f - add wheel hashes to the requirements file`. This made it possible to implement security and maintainability solutions which sometimes felt like an overkill.

When it came to the use of GAI, ChatGPT and Github Copilot was used to better understand coding errors and thereby helping in solving them. Another aspect of using AI, would be on how to do specific things in different languages. For example "How do i write inline code in .md?" or "How do i change the rejection status code on the rate limiter?".

The **Google Gemini 3 model** has primarily been used for debugging and to resolve technical uncertainty during development.

This includes asking the generative AI model to:

- find possible fixes for errors, as well as explaining how and why they appeared.
- provide an overview and understanding of important features and commands of new tools and technologies.
- review developer decisions to ensure that any changes to the application is correct.
- check for grammar and phrasings when writing documentation or report.